



Kang YeonBae
AI Full-stack Engineer

APPLIED AI · BACKEND · PRODUCT

복잡한 문제를 운영되는 제품으로.

AI·백엔드·제품 경험을 연결해 아이디어를 실제 업무와 사용자가 계속 사용할 수 있는 서비스로 만듭니다.

OPERATIONS

40%

VOC/CS 생산성 개선

MODEL

95%+

리뷰 긍·부정 분류 정확도

DELIVERY

8

요구 분석부터 운영한 실무 제품

ENGINEERING PROFILE

기능을 구현하는 데서 끝나지 않고,
배포 이후의 운영까지 책임집니다.

EXPERIENCE

2025.07 - NOW

Hurdlers · AI Engineer

광고·커머스·VOC를 위한 AI 제품과 업무 자동화

2025.03 - 2025.07

Mooker · Backend Lead

폐쇄망 DID CMS 백엔드와 미디어 아키텍처

2023 - 2024

Codelab · SeSac

클라우드·AICC·백엔드 집중 과정, 최우수 프로젝트

CORE IMPACT

PROBLEM

반복 광고 분석 · 대량 입찰 탐색 · 복잡한 유료 콘텐츠 경험

MY ROLE

문제 정의부터 AI·Backend·Frontend·Cloud 배포까지 직접 책임

RESULT

근거 검수 자동화 · 운영 중인 입찰 서비스 · 실결제 B2C 제품

HOW I BUILD

01 문제를 좁힙니다

기술보다 먼저 반복 비용과 실패 경계, 실제 사용자를 확인합니다.

02 맞는 크기의 기술을 고릅니다

규칙·전통 ML·LLM·Agent를 정확도와 비용에 맞춰 조합합니다.

03 운영 가능하게 배포합니다

로그, 검증, 재시도, 권한과 사람이 개입할 지점을 함께 설계합니다.

CASE STUDY 01 · PRODUCT & SYSTEM

Amazon Ads Reporting Agent

리포트를 쓰는 LLM과, 그 숫자를 검증하는 Python을 분리했습니다.

PERIOD	ROLE	COLLABORATION	STATUS
2025.10 – 2026.05	기획 · AI · Backend · Cloud / 단독 개발	클라이언트 요구 분석 · 마케터와 AI 역할 경계 합의	운영 중

PRODUCT EVIDENCE

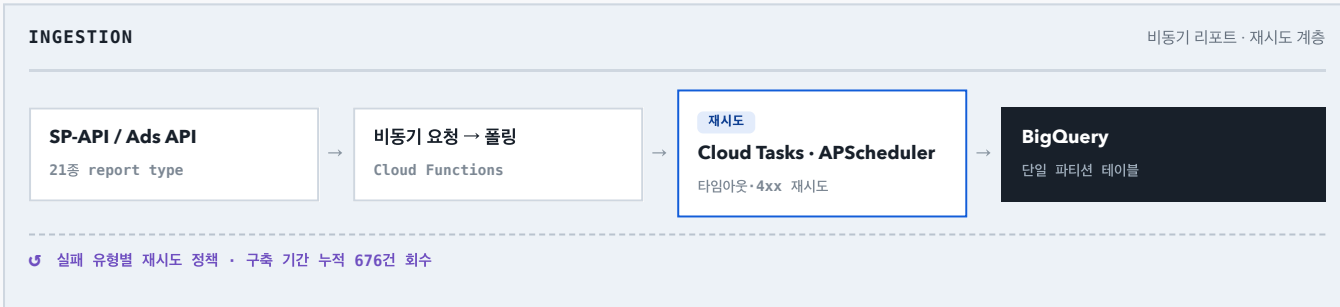
BigQuery 실측 · 2026.08 기준

1억 3,500만 행 누적 적재 약 29.7GB	21,607건 누적 수집 작업 21종 report type	319일 광고 성과 커버 2025.09.23 – 2026.08.09
12개사 / 41키 멀티테넌트 sp_api 15 · advertising 14 · ads_v3 12	239 → 1 테이블 마이그레이션 날짜 사당 → 단일 파티션	296개 관리 캠페인 성과 레코드 48,988행

단일 최대 테이블은 brand_analytics_data 1억 2,952만 행 / 28.7GB입니다. 날짜 사당(239개 사드) 구조를 단일 파티션 테이블로 마이그레이션해 쿼리 경로를 단순화했습니다.

ARCHITECTURE

LLM 생성 / Python 검증 분리



RESULT

운영 테넌트별 주간·월간 리포트를 자동 생성해 운영 중입니다. 최근 기준 하루 26~54개 수집 job이 돌고, 산출물은 주간 3~4분 / 월간 4~8분에 생성됩니다.	검증 가능성 리포트의 모든 금액은 BigQuery 원본 행으로 만든 표의 값 집합과 대조해 통과시킵니다. 모델이 만들어낸 숫자는 문서에 남지 않습니다.	대외 구축 사례가 동아일보 경제면에 보도됐습니다. 리포트가 다루는 광고 집행 규모는 누적 광고비 \$463,984 · 매출 \$734,727(ROAS 1.58)입니다.
---	--	---

Google ADK

Gemini

BigQuery

Cloud Tasks

Cloud Functions

Amazon SP-API

[동아일보 경제면 보도](#)

ENGINEERING DECISIONS

무엇을 고르지 않았는지가 설계를 설명합니다.

01 에이전트 자율 라우팅을 폐기했습니다

검토	ADK Coordinator가 리포트 5개 섹션을 자율 라우팅하는 multi-turn 구조로 먼저 구현했습니다.
문제	같은 입력이 실행마다 다른 경로를 탔습니다. 실패를 재현할 수 없으니 디버깅이 성립하지 않았고, 리포트 품질 편차의 원인을 특정할 수 없었습니다.
결정	2026-05-14 재설계로 오케스트레이션을 Python 직선 흐름으로 되돌렸습니다. ADK는 LLM 호출 래퍼로만 씁니다.
결과	Vertex 네이티브 인증(ADC), tool calling 추상화, before/after_model_callback 훅(토큰 사용량 로깅·tool_config 스왑), 세션 state는 그대로 활용합니다.

프레임워크 선택의 문제가 아니라, 프로덕션에서 자율성을 어디까지 허용할지의 문제였습니다. 재현 가능성을 포기하는 자율성은 쓰지 않습니다.

02 검증자를 LLM으로 두지 않았습니다

검토	생성 결과를 다른 LLM이 심사하는 LLM-as-judge 구성이 일반적인 선택이었습니다.
문제	검증자를 LLM으로 두면 환각을 환각이 승인합니다. 조작된 숫자와 바뀐 캠페인명을 걸러낼 근거가 없습니다.
결정	생성(LLM) → fact_guard(Python) → 교정(LLM) → report_qa_agent(Python)의 3계층으로 나누고, 두 검증 계층을 전부 결정론적 Python으로 구현했습니다. 달러 금액은 BigQuery 원본 행으로 만든 표의 값 집합에 없으면 통과하지 못합니다.

LLM은 서사를 고르고, 숫자의 진위는 코드가 판정합니다. 역할을 섞지 않는 것이 신뢰도의 전제입니다.

03 AI의 역할을 '현상 정리'로 한정했습니다

검토	재고·시즌성·프로모션 맥락까지 LLM이 추론해 전략을 제안하는 방향도 검토했습니다.
문제	데이터 밖 맥락을 모델이 추정하기 시작하면 리포트 전체의 신뢰도가 흔들립니다. 틀린 전략 제안 한 줄이 맞는 분석 열 줄을 무효로 만듭니다.
결정	AI는 지출 변화·ROAS 증감처럼 데이터로 확인되는 현상만 정리하고, 최종 전략 판단은 마케터가 말도록 역할을 분리했습니다. 이 경계는 클라이언트·마케터와 합의해 설계에 반영했습니다.

성급한 결론 리스크를 프롬프트가 아니라 협업 구조로 막았습니다.

This is intentionally not an LLM judge. The final report is checked against the Python-built tables and raw DB rows that came from BigQuery, so another model cannot approve hallucinated numbers or rewritten campaign names.

report_qa_agent.py - 코드에 남긴 설계 의도

TROUBLESHOOTING & COLLABORATION

무엇이 깨졌고, 누구와 무엇을 합의했는지.

TROUBLESHOOTING

가장 오래 붙잡은 문제

LLM 환각 – 숫자와 용어의 조작

실제 전환이 있는데도 '직접 매출 없음'으로 서술된 고객 피드백 P0를 계기로, 생성 결과를 원본 데이터와 대조하는 출고 차단선을 세웠습니다.

2026-06-04 · 고객 피드백 P0

증상 Cross-sell 섹션에 direct conversion이 실제로 존재했는데 "직접 매출 없음"으로 서술 – 실제 성과와 정반대의 문장이 나왔습니다.

대응 fact_guard에 _DIRECT_ABSENCE_CLAIM_RE 정규식을 추가해 같은 유형의 서술을 생성 단계에서 차단했습니다.

report_qa_agent의 금치어 규칙에는 source_type, served_search_term, Drill-Down, Top of Search, 구매 여정, 후광 효과 등 수십 개가 쌓여 있습니다. 한 번에 설계한 규칙이 아니라 반복 실패가 남긴 화석입니다.

수집 계층 – 변경 중인 외부 API 위에서의 수집 안정화

구축 시점의 Amazon 리포트 API는 개편이 진행 중이었습니다. 콘솔 화면과 응답 스펙이 함께 바뀌었고 참고할 선례가 없어, 실패 유형을 하나씩 관측해 분류하는 방식으로 대응했습니다.

SP-API Unauthorized (토큰 만료)	최다
catalog 200 응답 에러	143건
Inventory 리포트 FATAL	73건
Sales 3일 초과 타임아웃 · 404	59건
Brand Analytics FATAL	26건

실패 유형별로 재시도 정책을 나눈 결과, 구축 기간 누적 21,607건 중 676건을 재시도로 회수했습니다. 남은 영구 실패 373건은 대부분 API 스펙을 역설계하던 PoC 구간에 몰려 있습니다.

COLLABORATION

요구를 수용 기준으로 번역하기

클라이언트사의 마케터가 실제 사용자였습니다. 요구는 "리포트를 자동으로 만들어 달라"였지만, 업체마다 리포트 형식과 판단 기준이 달라 그대로는 자동화할 대상이 정의되지 않았습니다.

클라이언트가 말한 것	내가 설계로 옮긴 것
"업체마다 리포트가 달라서 비교가 안 됩니다."	기존 리포트를 직접 수집·분석해 공통 항목을 추려 표준 리포트 생성 기준을 정립했습니다. 자동화 이전에 문서 규격을 먼저 합의했습니다.
"AI가 전략까지 제안해 주면 좋겠습니다."	재고·시즌성·프로모션은 데이터에 없어 모델이 추정하면 틀립니다. AI는 현상 정리까지, 전략 판단은 마케터가 맡는 것으로 역할 경계를 합의했습니다.
"Cross-sell 항목이 실제와 반대로 쓰여 있습니다." (2026-06-04 · P0)	제발 방식을 프롬프트가 아니라 결정론적 규칙으로 잡겠다고 정하고, fact_guard에 해당 서술 패턴을 차단하는 검사를 추가했습니다.

사람이 개입할 지점을 남겨 두었습니다

완전 자동화 대신, AI가 만든 문장의 근거가 되는 표와 원본 수치를 리포트에 함께 실었습니다. 마케터가 결론을 검토할 수 있어야 자동화를 신뢰하고 계속 쓴다고 판단했습니다.

CASE STUDY 02 · PRODUCT & SYSTEM

Bid Master

여러 사이트를 긁는 대신, 폐쇄형 조달 시스템 한 곳을 API 레벨까지 역설계했습니다.

PERIOD	ROLE	COLLABORATION	STATUS
2025.11 – 2025.12	제품 흐름 설계 · Full-stack · 배포 / 단독 개발	기획은 클라이언트 · 회의록 기반 수용 기준 합의	운영 중

PRODUCT EVIDENCE

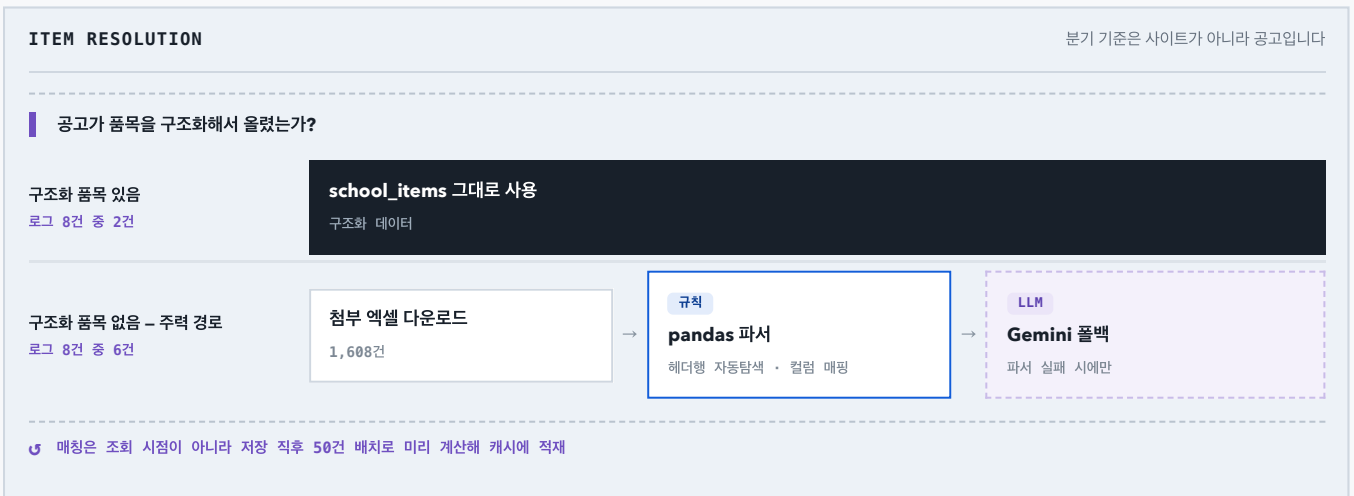
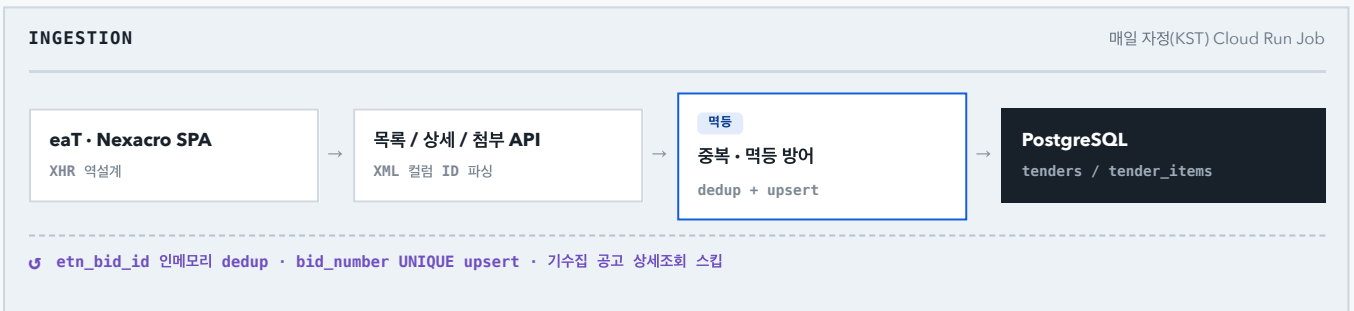
운영 DB 덤프 실측 · 2026.03.06 기준

3~4시간 → 30분 실사용 소요시간 클라이언트 회의록 기준 · 약 85% 단축	877건 누적 수집 공고 일 자동 수집 · 어제자 진행중 8건 내외	29,761행 누적 품목 공고당 평균 34행
9,521개 정규화 품목 코드 대분류 4 · 중분류 59 · 품목그룹 4,229	1,416개교 학교 마스터 묵음입찰 개별학교 306행	1,608건 첨부파일 파싱 엑셀 자동 파서 + LLM 풀백

수집 대상은 eaT 학교급식전자조달 1곳이고, 역설계한 API 엔드포인트는 목록·상세·첨부 3종입니다. 지역코드 250개를 파싱해 보유하되 운영은 서울·경기 2개 지역 × 축산 3개 키워드, 하루 6 조합으로 좁혀 실행합니다.

ARCHITECTURE

수집 → 품목 해석 분기



RESULT

업무 매월 20일 전후 5~6일에 물량이 물리는 업무입니다. 엑셀로 3~4시간 걸리던 검토가 30분 내외로 줄었다는 것이 클라이언트 회의록에 기록돼 있습니다.	자동화 범위 수집(자정 자동 실행) · 매칭(저장 직후 사전 계산) · 판단(하한가 = 단가 × 사정률, 예상이익 = 하한가합계 – 내물 품가) 세 단계에서 수작업을 제거했습니다.	데이터 자산 공고를 모으는 데서 끝내지 않고 9,521개 표준 식품코드로 정규화해, 품목·단가를 누적 비교할 수 있는 DB를 만들었습니다.
--	--	---

ENGINEERING DECISIONS

무엇을 고르지 않았는지가 설계를 설명합니다.

01 브라우저 자동화에서 API 직접 호출로 전환했습니다

검토	대상이 Nexacro 기반 SPA라 HTML 소스에 데이터가 없었습니다. HTML 파서로는 파싱할 대상 자체가 없어 후보에서 먼저 제외했습니다.
문제	그래서 Playwright로 브라우저를 띄워 클릭 가능 요소를 스캔·좌표 매핑(SoM)해 조작했습니다. 다만 화면 DOM 변경에 취약했고, Cloud Run Job에서 브라우저 기동 비용이 컸습니다.
결정	조작 과정에서 XHR을 관찰해 Nexacro XML 요청 스펙을 역설계하고, 목록·상세·첨부 3개 API를 직접 호출하는 순수 requests 파이프라인으로 전환했습니다.
결과	브라우저 기동이 사라져 Cloud Run Job(2Gi, asia-northeast3)에서 안정적으로 돌고, XML 컬럼 ID 기반 파싱이라 화면 변경에 둔감해졌습니다.
남은 부채	Dockerfile에 chromium 설치가 남아 이미지가 불필요하게 큼니다. 전환은 마쳤지만 정리하지 못한 부채로 인지하고 있습니다.

자동화 도구를 고르는 문제가 아니라, 조작해야 할 화면을 관측 가능한 계약으로 바꾸는 문제였습니다.

02 매칭을 조회 시점이 아니라 저장 직후에 계산했습니다

검토	목록을 열 때 사용자별 수익성을 실시간으로 계산하는 방식이 기본 선택이었습니다.
문제	입찰 시즌이 매월 20일 전후 5~6일에 몰립니다. 사용자가 가장 몰리는 시점에 가장 무거운 계산이 겹치는 구조였습니다.
결정	수집 저장 직후 calculate-matches를 50건 배치로 트리거해 user_tender_matches 캐시에 미리 계산해 두고, 목록 조회는 캐시만 읽도록 분리했습니다.

사용자가 몰리는 시점과 계산이 가능한 시점이 다르면, 계산을 앞으로 당깁니다.

03 자동화가 불가능한 지점은 사용자 참여로 바꿨습니다

검토	G마크 요구 학교는 정보가 엑셀이 아니라 한글 파일로 오는 경우가 있어 자동 추출이 불가능했습니다.
문제	완전 자동화를 고집하면 해당 학교는 아예 다룰 수 없게 되고, 사용자는 다시 수작업으로 돌아갑니다.
결정	자동 추출을 포기하고, 사용자가 한 번 체크하면 다음부터 배치로 표시되는 토글로 바꿨습니다. 단가 DB와 공유 단가 DB도 같은 방식으로 설계했습니다.

자동화 실패를 사용자 참여형 데이터 자산으로 바꿨습니다. 한 번의 입력이 다음 사용자에게 남습니다.

TROUBLESHOOTING & COLLABORATION

무엇이 깨졌고, 누구와 무엇을 합의했는지.

TROUBLESHOOTING

가장 오래 붙잡은 문제

학교마다 제각각인 엑셀 양식

실제 변동은 사이트 구조가 아니라 첨부 파일이었습니다. API 경로는 XML 컬럼 ID 기반이라 화면 변경에 둔감했고, 깨지는 쪽은 항상 학교가 직접 올린 엑셀이었습니다.

다중 시트 공고

증상	한 학교가 농산·축산·수산물 시트로 나눠 올린 경우, 전체 시트를 읽어 매칭률과 예상 금액이 함께 왜곡됐습니다.
대응	키워드와 시트명을 매칭해 대상 시트만 파싱하고, 매칭되는 시트가 없으면 전체로 폴백합니다. 그리고 실제로 읽은 시트명을 UI에 노출해 사용자가 확인할 수 있게 했습니다.

파싱은 헤더행 자동 탐색 → 컬럼명 매핑 → 인덱스 컬럼 판별 순으로 시도하고, 모두 실패할 때만 Gemini로 넘깁니다. LLM을 기본 경로가 아니라 마지막 폴백에 둔 이유는 비용과 재현성 때문입니다.

중복·재실행·유실 방어

지역별로 순회하는 구조라 같은 공고가 여러 번 수집됩니다. 중복 제거와 재실행 안전성을 계층으로 나눠 처리했습니다.

etn_bid_id 인메모리 dedup	1차
bid_number UNIQUE + upsert	역동
기수집 공고 상세조회 스킴	LLM 비용
네트워크 오류만 선별 백오프	5~10~20초
시작 전 DB 연결 확인	5회

저장이 불가능한 상태에서 크롤링하면 수집한 데이터가 그대로 유실됩니다. 그래서 크롤링을 시작하기 전에 DB 연결을 백오프로 확인하고, 실패하면 아예 시작하지 않습니다. 남은 과제는 엑셀 경로 품목의 school_id가 NULL로 남는 케이스입니다 – tender_schools가 API 품목 경로에서만 insert되는 구조가 원인입니다.

COLLABORATION

요구를 수용 기준으로 번역하기

기획을 클라이언트가 맡은 프로젝트였습니다. 요구는 대부분 증상으로 전달됐고, 저는 그것을 가설과 수용 기준으로 번역하는 역할을 했습니다.

클라이언트가 말한 것	내가 설계로 옮긴 것
“학교를 6번째 연속으로 클릭하면 데이터가 안 나옵니다.”	요청 레이스 컨디션 · 세션 만료 · 요청 취소 미처리 · 429 · 전역 상태 덮어쓰기로 가설을 나눴습니다. 수용 기준은 “20회 연속 클릭에도 미재현” 그리고 “빈 화면 대신 재시도·원인 메시지·자동 재요청 중 최소 하나는 동작”으로 합의했습니다.
“뒤로 가면 데이터가 없어집니다. 될 때도 있고 안 될 때도 있고요.”	상태 소실로 가설을 세우고, “검색어·필터·스크롤 위치·결과 리스트 100% 복구, 브라우저 뒤로가기와 앱 내 뒤로가기 동일 동작”을 수용 기준으로 문서화했습니다.
“축산 시트만 읽어주세요.”	첨부 구조상 사용자가 시트를 고를 수 없는 케이스가 있었습니다. 자동 판별 규칙을 적용하되 현재 읽은 시트명을 화면에 표시하는 것으로 합의했습니다. 완전 자동화 대신 관측 가능성으로 신뢰를 확보한 경우입니다.

우선순위를 근거를 갖고 합의했습니다

클라이언트가 “안정성만 잡히면 유료로 쓰겠다”고 말한 것을 근거로, 신규 기능보다 P0 안정성 버그를 먼저 처리하기로 정했습니다. 실제로 P0 8건을 끝낸 뒤 기능 6건 순서로 진행했습니다.

CASE STUDY 03 · PRODUCT & SYSTEM

Byeoljari.com

성공 기준을 '결제 완료'가 아니라 '결제한 사람이 결과를 받는 것'으로 정의했습니다.

PERIOD	ROLE	COLLABORATION	STATUS
2026.03 – 운영 중	기획 · 디자인 · Frontend · Backend · Infra / 단독 개발	외부 코드 리뷰 수용 · 사업자 등록 및 PG 계약 직접 수행	실결제 운영 중

PRODUCT EVIDENCE

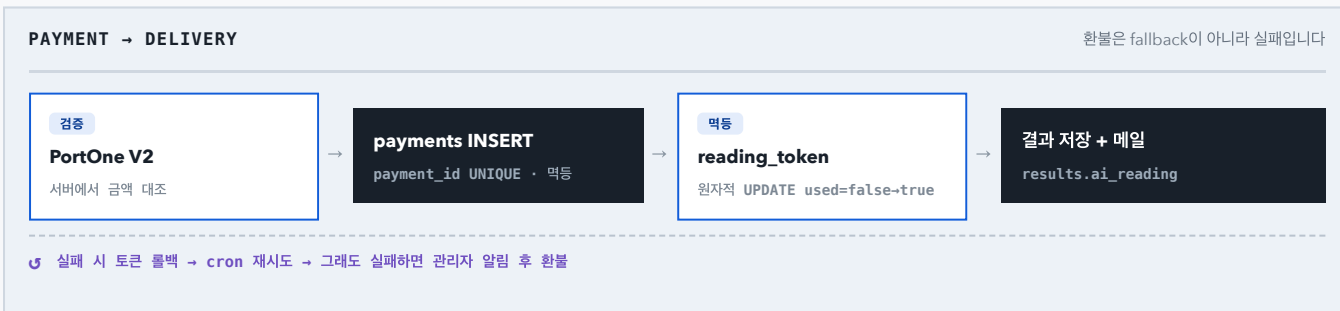
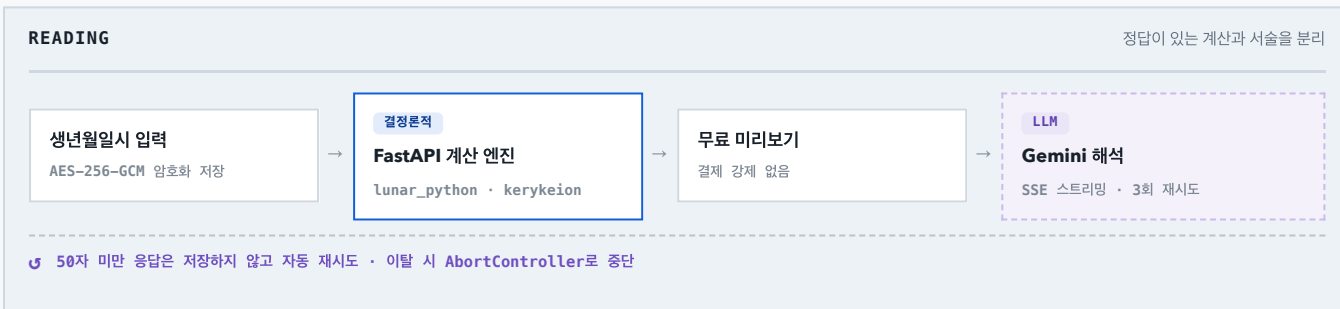
실서비스 운영 · 사용자 시나리오 검증

실결제 운영 B2C 서비스 사업자 등록 · PG 가맹점 계약	기획 → 운영 전 과정 단독 책임 제품 · 디자인 · 개발 · 배포	무료 → 유료 전환 흐름 기본 결과 후 AI 상세 리딩
11종 유료 · 무료 서비스 사주 · 점성술 · 타로 · 궁합 · 일일운세	24개 핵심 사용자 시나리오 결제 · 결과 전달 · 환불 검증	유입 → 결제 직접 운영한 퍼널 SEO · Meta 광고 · PG

결제는 PortOne V2로 카카오페이(회원)와 토스페이(비회원)를 모두 지원합니다. 가격은 코드 상수가 아니라 service_prices 테이블이 유일 권위 소스이며, 관리자 화면에서 배포 없이 변경합니다.

ARCHITECTURE

결정론적 계산 / AI 해석 분리



RESULT

제품 기획·디자인부터 프론트엔드·백엔드·인프라까지 단독으로 만들어 실결제가 일어나는 B2C 서비스로 운영 중입니다. 사업자 등록과 PG 가맹점 계약도 직접 진행했습니다.	무결성 결제 금액 서버 대조, payment_id 역등성, 낙관적 잠금, 토큰 원자적 소비, 타인 결제 환불 차단을 코드 레벨에서 강제합니다. 감사에서 나온 CRITICAL 7건·HIGH 10건을 모두 해소했습니다.	운영 가격을 DB 권위로 두어 광고 집행 중에도 배포 없이 조정할 수 있고, 관리자 화면에서 매출·사용자·리딩 현황을 확인합니다. Meta 광고를 직접 집행해 유입부터 결제까지의 퍼널을 봤습니다.
--	--	---

Next.js 16 · React 19 · FastAPI · Supabase · PortOne V2 · Gemini · Vercel

byeoljari.com >

ENGINEERING DECISIONS

무엇을 고르지 않았는지가 설계를 설명합니다.

01 도메인 계산을 LLM이 아니라 Python으로 분리했습니다

검토	생년월일을 그대로 LLM에 넘겨 명식 산출과 해석을 한 번에 시키는 구성이 가장 빠른 길이었습니다.
문제	만세력 변환-절기 경계-행성 위치는 정답이 있는 계산입니다. 모델이 추정하면 같은 생년월일에 다른 결과가 나오고, 유료 서비스에서 그건 제품 실패입니다.
결정	lunar_python과 kerykeion을 쓰는 FastAPI 계산 엔진을 분리해 명식을 결정론적으로 산출하고, Gemini는 산출된 명식을 해석하는 역할만 맡깁니다.

정답이 있는 것은 코드가 계산하고, 서술이 필요한 것만 모델에 맡깁니다. Amazon Ads Agent에서 검증을 Python에 둔 것과 같은 원칙입니다.

02 성공 기준을 '결제 완료'가 아니라 '결과 수신'으로 잡았습니다

검토	결제 검증까지 책임지고 AI가 실패하면 환불하는 구조가 일반적인 선택이었습니다.
문제	환불은 사용자 입장에서 실패입니다. 돈은 돌아오지만 원했던 결과는 받지 못합니다. 게다가 토큰을 먼저 used=true로 잠그는 구조에서는, AI 호출 중 런타임이 끊기면 그 주문이 재시도 대상에서 아예 빠집니다.
결정	'환불은 fallback이 아니라 실패'를 기준으로 삼고, 토큰 롤백 · cron 재처리 · 관리자 알림을 붙여 결제된 주문이 결과에 도달할 때까지 추적되도록 만들었습니다.
남은 부채	게스트 결제는 webhook이 결과의 writeToken을 검증할 수 없어 클라이언트 verify에 의존이 남아 있고, vercel.json에는 cron 스케줄 정의가 없습니다. 저장소 기준으로 자동 복구가 주기 실행된다는 보장은 아직 없습니다.

성공 기준을 어디에 두느냐가 아키텍처를 결정합니다. 결제가 아니라 전달을 기준으로 잡으면 필요한 상태가 달라집니다.

03 가격을 코드가 아니라 DB에 두었습니다

검토	가격을 코드 상수로 관리하는 방식이 단순하고 타입 안전합니다.
문제	가격을 바꿀 때마다 배포가 필요합니다. 광고를 집행하는 중에는 가격 실험 자체가 불가능해집니다.
결정	service_prices 테이블을 유일 권위 소스로 두고 관리자 화면에서 변경합니다. 코드의 상수는 타입과 초기값 전용으로만 남겼습니다.

사업 판단으로 자주 바뀌는 값은 배포 경로에서 빼냅니다.

성공 기준 : 결제 → 사용자가 제대로된 리딩 결과를 받는다. 환불은 fallback이 아니라 실패.

CLAUDE.md - 프로젝트 문서 첫 줄에 적어둔 성공 기준

TROUBLESHOOTING & COLLABORATION

무엇이 깨졌고, 누구와 무엇을 합의했는지.

TROUBLESHOOTING

가장 오래 붙잡은 문제

환불 계산이 금융 손실로 이어질 수 있었던 문제

사용자 시나리오 검증 중 결제 식별자와 사용자의 연결 누락을 발견해, 결제 SDK에 식별자를 심고 서버에서 다시 대조·차단하도록 바꿨습니다.

2026-05-21 · P0-A

증상 회원이 외부결제로 콘텐츠를 사면 충전과 구분되지 않는 PAID 행이 남습니다. FIFO 잔여 계산이 이를 충전금으로 취급해, 이미 100% 소진된 14,900원 결제에 환불 가능 잔액 10,000원이 잡혔습니다. 서비스도 받고 환불도 받는 경로였습니다.

대응 charge-and-use 결제를 충전형과 구분해 기록하고, FIFO 계산이 use 트랜잭션의 chargePaymentId까지 보도록 고쳤습니다. 어느 결제가 이미 소진됐는지를 계산이 알 수 있게 만든 것이 핵심입니다.

기존 결제와 데이터 구조를 깨지 않으면서 신규 결제부터 식별자 검증을 적용했습니다. 결제 로직에서는 빠른 수정 자체보다 기존 주문의 호환성과 결과 전달 순서를 지키는 것이 중요했습니다.

결제 · 리딩 무결성 방어

결제 성공 화면이 아니라 사용자가 결과를 받는 순간까지를 하나의 시나리오로 보고, 결제·토큰·리딩·환불 경계를 반복 검증했습니다. 아래는 그 과정에서 담은 대표 실패 경로입니다.

결제 금액 클라이언트 변조	서버 대조
reading_token 이중 사용	원자적 UPDATE
charge TOCTOU 레이스	UNIQUE + 23505
게스트 결제 금액 무제한	상한 검증
returnUrl open redirect	상대경로만

CRITICAL 7건과 HIGH 10건을 모두 해소했습니다. 남은 과제는 게스트 결제의 **webhook** 자동 보강과 **cron** 스케줄 정의이며, 문서에 그대로 기록해 두었습니다.

REVIEW

혼자 만든 제품에서 검증을 확보하는 법

클라이언트도 리뷰어도 없는 1인 제품이었습니다. 요구를 주는 사람과 검증하는 사람이 모두 저라는 것이 가장 큰 리스크라고 보고, 외부 코드 리뷰를 의도적으로 끌어들여 검증 경로를 만들었습니다.

리뷰에서 지적받은 것	내가 검증하고 고친 것
“회원 charge-and-use 결제가 환불 pool을 오염시킨다.”	지적을 그대로 수용하지 않고 코드로 시나리오를 재현해 실제 금융 손실 경로임을 먼저 확인했습니다. 그 과정에서 지적되지 않은 결함 2건을 추가로 찾아 함께 고쳤습니다.
“cron 라우트는 있는데 스케줄이 보장되는지 모르겠다.”	vercel.json에 crons 정의가 없다는 사실만 확인하고, 외부 대시보드에서 돌고 있을 가능성은 추측으로 적지 않았습니다. “저장소 기준으로는 보장이 없다”고 문서에 그대로 남겼습니다.
“결제를 강제하면 유입이 이탈한다.” (광고 집행 결과)	온보딩과 즉시 결제를 강제하던 랜딩을 접고, 무료 기본 결과를 먼저 보여준 뒤 AI 상세 리딩만 결제로 유도하는 흐름으로 바꿨습니다. 기존 랜딩은 삭제하지 않고 복구 절차와 함께 보관했습니다.

판단 근거를 운영 원칙으로 남겼습니다

리뷰에서 배운 것을 개발 원칙으로 정리했습니다. 증상을 감추는 예외 처리보다 원인을 제거하고, 문제가 생긴 뒤의 처리보다 발생 조건을 먼저 검증한다는 기준입니다. 다음 변경에서도 같은 실패를 반복하지 않기 위한 제품 운영 규칙입니다.

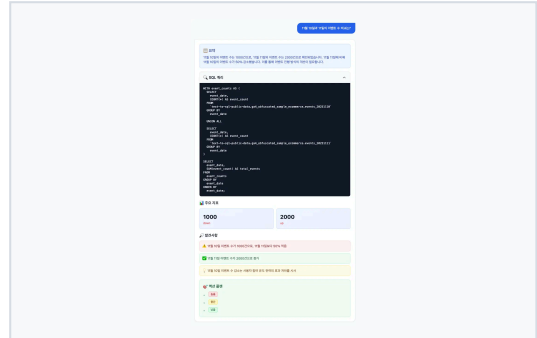
ROLE-SPECIFIC EVIDENCE

데이터 질문과 실시간 회의를 제품 흐름으로 연결합니다.

APPLIED AI

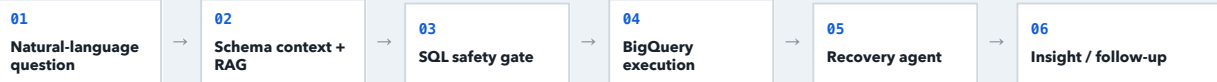
BigQuery AI Analyst

자연어 질문을 스키마에 맞는 SQL로 만들고, 안전성 검사·실행·오류 복구·인사이트 생성까지 연결했습니다.



SYSTEM FLOW

INPUT → RESULT



MY ROLE

AI · Backend · Data Product / 단독 개발

RESULT

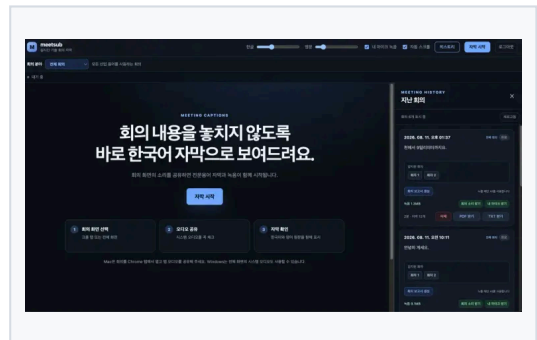
SELECT/WITH only · 최대 2회 복구 · SSE 진행 상태

Google ADK · LangGraph · BigQuery · RAG

REAL-TIME PRODUCT

MeetSub

설치가 제한된 회의 환경에서 브라우저 탭 오디오를 받아 전문 기술용어가 정확한 한영 자막을 제공합니다.



SYSTEM FLOW

INPUT → RESULT



MY ROLE

Product · Full-stack · Cloud / 단독 개발

RESULT

무설치 브라우저 · 전문용어 교정 · 읽은 자막 안정화

Deepgram · Gemini · WebSocket · Cloud Run [LIVE SITE](#)

ONE CLIENT · TWO OPERATIONAL PROBLEMS

상담 정리와 리뷰 판독을 연결해 CS 운영 비용을 함께 줄였습니다.

PROBLEM

같은 고객사에서 상담 기록과 상품 리뷰가 빠르게 쌓였고, 사람이 다시 읽어 정리하는 반복 업무와 범용 AI API 비용이 함께 증가했습니다.

MY ROLE

두 제품 모두 요구 분석, 기획, 설계, 개발, 패키징과 배포까지 전 과정을 단독으로 담당했습니다.

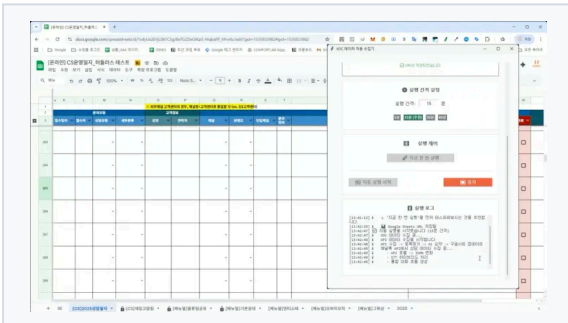
RESULT

상담 기록 정리 자동화로 CS 생산성을 약 40% 개선했고, 자체 학습 모델로 리뷰 긍정·부정 분류 정확도 95% 이상을 달성했습니다.

01 · LLM OPERATIONS

VOC Counseling Analyzer

채널톡 상담을 수집해 LLM으로 요약·항목화하고 Google Sheets 리포트까지 자동 생성했습니다.



SYSTEM FLOW

INPUT → RESULT



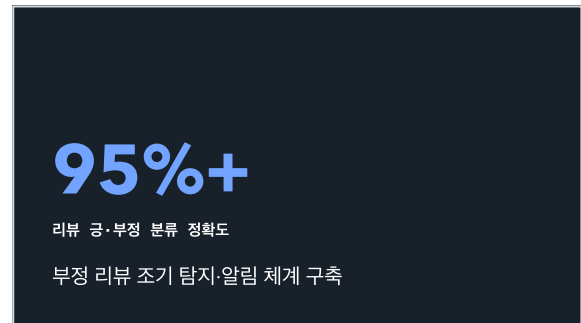
성과 · VOC/CS 반복 업무 약 40% 절감

OpenAI · ChannelTalk API · Streamlit · PyInstaller

02 · RIGHT-SIZED MODEL

Review Sentiment Classifier

AI-Hub 데이터로 TF-IDF·Linear SVM 기반 경량 NLP 모델을 직접 학습하고, 리뷰 수집·판독·부정 리뷰 알리를 연결했습니다.



SYSTEM FLOW

INPUT → RESULT

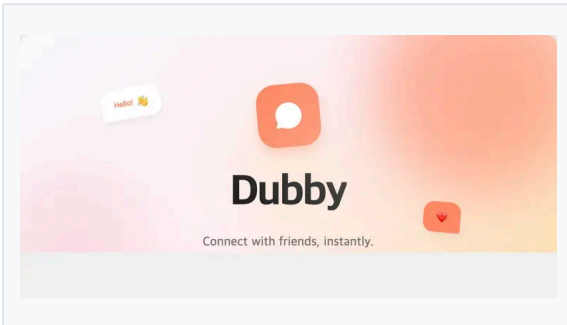


성과 · 반복 분류의 외부 API 의존과 비용 축소

scikit-learn · TF-IDF · Linear SVM · Flask

PRODUCT BREADTH

도메인이 달라도, 문제를 제품으로 끝까지 연결합니다.



MOBILE PRODUCT

Dubby

1:1 그룹 채팅, 스토리, 투표와 서버 권위형 카드게임을 하나로 묶어 Google Play에 출시

Flutter · Firebase · Cloud Functions

[GOOGLE PLAY >](#)



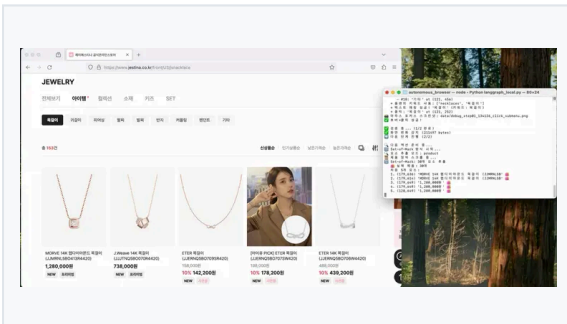
DOMAIN PRODUCT

Exam Forge

현직 강사의 불편에서 출발해 숫자·정답·풀이·TikZ 도형이 함께 바뀌고 A4로 바로 인쇄되는 시험지 PWA

Next.js · TikZ · KaTeX

[LIVE SITE >](#)

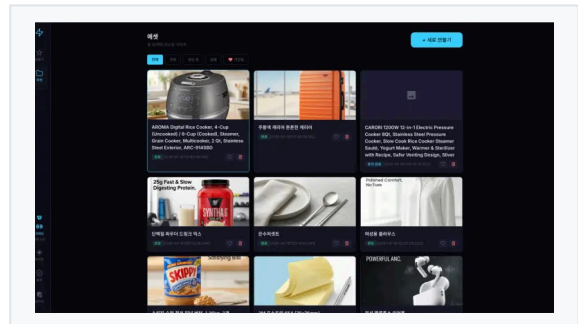


LOCAL MULTIMODAL

Local Browser Agent

고객 화면을 외부 API로 보내지 않고 자연어 목표를 실행하는 로컬 자율 브라우저 Agent

Qwen2.5 · LLaVA · Playwright



CREATIVE AUTOMATION

Snap-p

상품 분석부터 아마존 리스팅 이미지 9장·카피·정책 검수까지 자동화

Gemini · Next.js · Prisma

[LIVE SITE >](#)

STUDENT FOUNDATION

팀 프로젝트에서 시작해, 운영 제품을 만드는 엔지니어로.



Echo Recycle Hub

폐기물 이미지 분류를 배출 방법·재활용 추천·챗봇·음성 상담까지 연결한 AICC 서비스.

Architecture · Backend · ML / 기여도 50%

[GITHUB](#)



SpacePlace.store

공간 검색과 예약을 서비스 경계로 나누고 API 계약과 통합 비용을 경험한 MSA 팀 프로젝트.

Frontend · API / 기여도 35%

[GITHUB](#)

LET'S WORK TOGETHER

문제를 발견하고,
작동하는 제품으로 만들겠습니다.

EMAIL dusqo7951@gmail.com

GITHUB github.com/KangYeonbae

LINKEDIN [yeonbae-kang-973436334](https://www.linkedin.com/in/yeonbae-kang-973436334)

WEB ARCHIVE [19 selected projects](#)